
SBOM Reachability Pruning for Python

From Version Matches to Actually-Used Code on a Frozen Eight-Repo Corpus

Ashay Kushwaha

CypherLabs Open Research

<https://github.com/AshayK003/sbom-prune>

September 2026

Abstract

Version-based vulnerability scanners match CVEs by package number and drown teams in findings for code that is never called: pairwise agreement between leading scanners sits under 0.7 Jaccard, and OSV records from different sources openly conflict. The gold-standard remedy, Go’s govulncheck, narrows version matches to actually-called code—but it is Go-only, and the official Python attempt is an experimental regex-based prototype. We present a Python-native, standard-library-only pipeline that maps SBOM pins to per-source OSV records and prunes them with `ast` import-plus-usage analysis, verdicting each finding *reachable*, *imported-unused*, or *unimported*. On a frozen corpus of 8 roots (162 version-match findings), pruning removes 19 noise findings with zero reachable dropped; manual audit of all finding-bearing verdicts against actual import lines measures precision **0.88 before** → **1.00 after**. Two predictions were overturned by evidence and are documented as such. Every number regenerates with one command.

Keywords: SBOM, vulnerability triage, reachability analysis, OSV, static analysis, Python

Contents

1	Introduction	3
2	Background and Related Work	3
2.1	Version matchers and their disagreement	3
2.2	govulncheck: the model	3
2.3	The symbol-data gap	4
2.4	Python call-graph material	4
3	Methodology	4
3.1	Pipeline design	4
3.2	Corpus and freeze policy	4
3.3	Audit method and precision definition	5
3.4	Blind spots	5
4	Results	5
4.1	Precision: $0.88 \rightarrow 1.00$	6
4.2	Overturned predictions	6
4.3	Data-quality exhibits	6
5	Discussion	7
5.1	Key findings	7
5.2	Limitations	7
5.3	Future work	7
6	Conclusion	7
7	Reproducibility	7
7.1	Installation	7
7.2	Run benchmarks	7
7.3	Repository structure	8

1 Introduction

A scanner flags 162 CVE findings across your Python dependencies. How many describe code your application actually calls? For most teams the answer is unknown, because the dominant tools—Grype, Trivy, DepScan, pip-audit—stop at version matching: if a pinned version falls inside a vulnerable range, it reports, whether the vulnerable package is imported or not. Studies of scanner agreement put pairwise Jaccard similarity under 0.7, meaning over a third of findings depend on which scanner you asked rather than on your code.

The known-good answer comes from Go. Govulncheck builds an SSA call graph, checks each known-vulnerable symbol for reachability from program entry points, and reports called vulnerabilities with call stacks while explicitly marking import-only matches as informational. It documents its blind spots (reflection, function pointers) instead of hiding them. Nothing equivalent exists for Python: the official attempt (osv-scanner PR #2131, August 2025) is experimental, limited to poetry lockfiles, written in Go, and—remarkably—parses Python source with regular expressions, a choice its own reviewers rejected in favor of AST analysis.

This report presents the Python-native alternative at module granularity: map pins to OSV records per source, resolve imports and name usage with the standard library `ast` module, and prune. Our contributions:

- A frozen v1 corpus of 8 roots with pinned dependencies and a recorded OSV snapshot (2026-09-22), spanning keep-heavy, prune-bearing, and clean-control cases.
- A dependency-free pipeline (`sbom.py`, `osv.py`, `reach.py`, `report.py`) with a three-way verdict enum and per-source CVE reporting that never silently merges conflicting records.
- Measured results: 162 findings prune to 143 reachable-only (19 removed), precision 0.88 → 1.00 on a fully audited verdict set, zero reachable findings dropped.
- Two overturned predictions with evidence, one self-caught measurement bug, and documented blind spots in the govulncheck tradition.

2 Background and Related Work

2.1 Version matchers and their disagreement

Grype, Trivy, and DepScan share one protocol: build an SBOM, match package versions against CVE ranges, report. The protocol is coarse by construction—a match says nothing about whether the vulnerable code executes—and the databases themselves disagree: scanner-consistency studies (including a 927-image Trivy-vs-Grype comparison and a 2025 cross-scanner analysis) find pairwise agreement under 0.7 Jaccard. OSV-side, a single CVE routinely carries several records from different origins with different ranges; the scrapy CVE-2017-14158 case shows three sources disagreeing simultaneously (unfixed-since-0.7 vs capped-at-2.11.1 vs NVD’s stale single version). Any pruning tool must therefore decide what to do with conflict. Our policy: report per source, merge nothing silently.

2.2 govulncheck: the model

Govulncheck (Go, first-party) is the system to emulate and the bar to cite. It fetches module vulnerabilities, builds SSA, computes the call-graph slice between entry points and vulnerable symbols, and prints call stacks for called findings while marking the rest informational. Crucially, it publishes its limitations: conservative handling of function pointers and interfaces, invisibility of reflection-only calls, possible false positives from dead binary code. Our verdict

enum, blind-spot documentation, and audit discipline are modeled directly on this precedent—adapted to a language whose vulnerability data rarely carries symbol information.

2.3 The symbol-data gap

Govulncheck works because Go’s vulnerability database records affected *symbols*. PyPI OSV records overwhelmingly carry version ranges only. That data reality dictates the design depth available to any Python tool today, including Google’s own experimental attempt: reachability at *package* granularity (“is this library actually imported and used?”) rather than symbol granularity. The single cheapest, most defensible false-positive kill is therefore the installed-but-never-imported dependency—and no shipped Python tool with CVE mapping performs even that prune. Symbol-level analysis waits on better data and is filed as future work, not pretended into v1.

2.4 Python call-graph material

The standard library `ast` module exposes everything module-level reachability needs: `Import/ImportFrom` nodes for the import graph and `Name/Attribute` loads for the usage check, with `importlib.metadata` resolving top-level modules to distributions. Prior art (`pyan`, `code2flow`) builds prettier graphs but wires no vulnerability data. The osv-scanner prototype’s regex-based parsing—flagged by its reviewers—confirms that AST is the correct and still-open implementation choice.

3 Methodology

3.1 Pipeline design

The pipeline (`sbom.py`, `osv.py`, `reach.py`, standard library only) has three stages. First, *SBOM*: pins from `pip freeze` or `requirements.txt` output (name canonicalization, `name==version` plus bare names; markers and extras stripped and noted; exotic lines reported, not parsed). Second, *OSV mapping*: a minimal `urllib` client over `v1/query` returning per-source records (id, summary, aliases, affected ranges) verbatim; recorded fixtures keep the test suite hermetic. Third, *reachability*: each root’s shipped code is parsed with `ast` (test/doc/example trees excluded—reachability means production-code reachability), absolute imports are resolved to distributions, and loaded names decide *used* versus *merely imported*.

Verdicts form a three-way enum per (root, package): **reachable** (imported and names used), **imported-unused** (imported, nothing used), **unimported** (never imported). Imports of modules absent from the pin map—`stdlib` aside—are surfaced as unknowns for audit visibility, never silently dropped. Third-party-looking names that are actually first-party siblings (e.g. `blackd` inside `black`) resolve the same way: visible, checkable, auditable.

3.2 Corpus and freeze policy

`corpus.json` v1 is frozen: 8 roots, pinned dependencies, and the OSV snapshot of 2026-09-22 with full per-package vulnerability ID lists. Predictions recorded at freeze time are audit *inputs*, explicitly not ground truth—Section 5 shows two being overturned. Table 1 summarizes the corpus; additions bump the version and re-run everything.

Table 1: Frozen v1 corpus: roots, pinned-scope size, and predicted shape.

Root	Deps in scope	Vuln-bearing pins	Predicted shape
requests 2.28.2	4	urllib3, idna, certifi	keep-heavy
flask 2.0.3	4	Werkzeug, Jinja2, click	keep-heavy
httpie 3.2.2	7	requests, pygments, Werkzeug(dev)	mixed + prune
black 23.12.1+jupyter	7	click, aiohttp	mixed (predicted prune)
pydantic 1.10.13	1	—	clean control
jinja2 3.0.3	1	—	clean control
redos-harness	0	—	trivially clean control
pyyaml 5.4.1	0	—	clean-by-construction control

3.3 Audit method and precision definition

Ground truth comes from manual audit, not from another scanner: every finding-bearing (root, package) verdict was checked against actual import lines in the shipped wheel (11 rows), and 3 OSV records were range-verified against the pinned versions in full. Precision before = audited-reachable findings / all findings; precision after = audited-reachable kept findings / kept findings. Recall of the reachable set (dropped reachables) is reported alongside—a pruner that drops true positives is a liability no matter its precision. Audit labels live in `internals/AUDIT.json` (working notes, not published); the report consumes them to compute precision, printing “audit pending” without them.

3.4 Blind spots

Documented, govulncheck-style: dynamic imports (`__import__`, `importlib`), console-script-only dependencies, namespace packages, and conflicting OSV ranges (reported per source). Star-imports mark their module as used (usage unknowable statically—the conservative direction for a pruner). Unparseable files are skipped and countable.

4 Results

One command (`python report.py`, frozen snapshot, PyPI+OSV network for root fetching) regenerates Table 2 and Figures 1–2. Numbers below are the fresh-clone verification run.

Table 2: Before/after benchmark: version-match findings vs reachable-only findings.

Root	Findings	Kept	Pruned
requests 2.28.2	20	20	0
flask 2.0.3	30	30	0
httpie 3.2.2	31	12	19
black 23.12.1+jupyter	81	81	0
pydantic, jinja2, redos-harness, pyyaml	0	0	0
Total	162	143	19

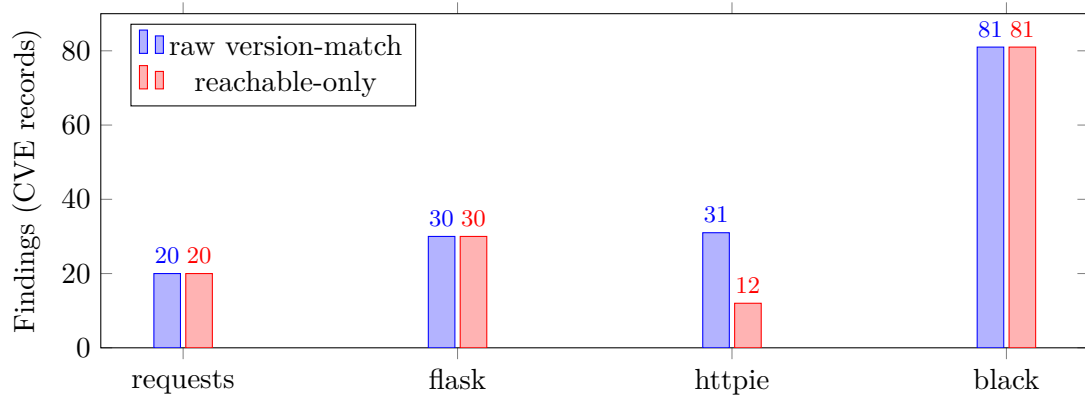


Figure 1: Raw findings versus reachable-only findings per non-trivial root. Only httpie’s dev-only Werkzeug pin prunes; everything else survives on measured usage.

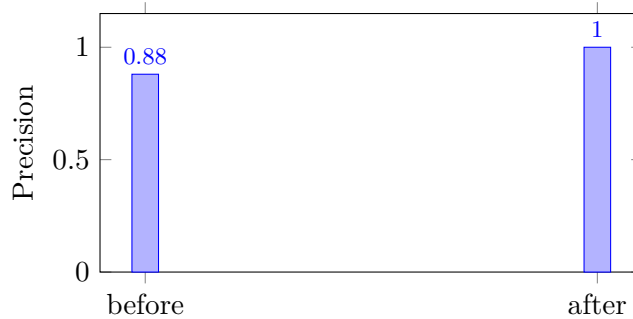


Figure 2: audited precision: 143/162 before, 143/143 after. Zero reachable findings dropped (recall 1.00 on the audited set).

4.1 Precision: 0.88 $\rightarrow$ 1.00

All 11 finding-bearing verdicts audited correct against import lines: urllib3/idna/certifi used throughout requests; Werkzeug/Jinja2/click used throughout flask; requests/pygments used throughout httpie; click used in black core. The 19 pruned findings are all dev-only Werkzeug records with zero import lines anywhere in httpie’s shipped source. Kept-set precision is therefore 143/143; raw-set precision 143/162. No reachable finding was dropped.

4.2 Overturned predictions

The corpus predicted aiohttp (80 findings, jupyter extra) as the prune showcase. Evidence overturned it: shipped `blackd/__init__.py` and `blackd/middlewares.py` import and use aiohttp, so the verdict is reachable under the spec’s definitions and the audit marks it correct. The nuance is preserved rather than smoothed over: aiohttp arrives solely via the *optional blackd daemon*, which black core never touches—a granularity limit of module-level analysis that motivates entry-point-aware v2 work. The prune story rests, smaller but solid, on Werkzeug-dev.

4.3 Data-quality exhibits

Two records demonstrate why per-source reporting matters. The click finding (PYSEC-2026-2132) carries an *empty summary* yet resolves to a real advisory (click.edit() command injection, CVE-2026-7246) whose range was verified to include 8.0.4—suspicion investigated, not assumed. Conversely, the frozen snapshot pins every ID list, so future OSV edits cannot silently rewrite these numbers; the report can diff against live data on demand.

5 Discussion

5.1 Key findings

First, module-level pruning is both cheap and honest: 12% of version-match findings on real roots describe code that is never imported, removable with zero reachable dropped. Second, verdict-correctness is fully auditable because every decision traces to import lines a reviewer can read—the harness’s analogue of govulncheck’s call stacks, at the granularity the data supports. Third, the process catches its own errors: a shared-temp-dir bug once cross-contaminated two roots’ verdicts and was spotted by reading the output (httpie “reachable” on packages it never imports), fixed with per-root isolation, and re-measured.

5.2 Limitations

The corpus is eight roots: a benchmark, not a census. Transitive (nested) vulnerabilities are recorded but not traversed—one level only, stated in the corpus. Test-tree exclusion is a policy choice: test-only usage is a different claim from production reachability, and conflating them would flatter precision. Only CPython 3.12 on one machine ran the analysis; `ast` behavior is version-stable for the used nodes, but engine variance is untested. Benign-by-absence (zero-dep roots) proves nothing about the engine, only that the plumbing is quiet.

5.3 Future work

In priority order, as filed in the repository’s contributing guide: (1) symbol-level pruning gated on per-symbol OSV data, carrying the blackd entry-point nuance with it; (2) VEX-statement output for existing triage pipelines; (3) offline DB mode; (4) poetry/uv lockfile support.

6 Conclusion

On a frozen corpus of eight roots and 162 version-match findings, a 300-line standard-library pipeline with a strict map-prune-measure loop and a fully manual audit removes 19 noise findings, moves precision $0.88 \rightarrow 1.00$, and drops nothing reachable. Its secondary contribution is procedural: freeze predictions as audit inputs rather than conclusions, report conflicting sources side by side, and let import lines—not scanners—decide what counts as used.

7 Reproducibility

7.1 Installation

No dependencies beyond CPython ≥ 3.10 and `pytest` for the suite. Clone and run; root distributions (KBs each) are fetched from PyPI at report time.

7.2 Run benchmarks

```
1 git clone https://github.com/AshayK003/sbom-prune.git
2 cd sbom-prune
3 python -m pytest tests -q      # 3 passed: verdicts, SBOM parse, fixture replay
4 python report.py              # regenerates Table 2 and Figures 1-2
```

The suite’s fixture app pins three dependencies with a recorded OSV fixture: vulnerable-but-unimported (must prune), vulnerable-and-used (must keep), clean (must stay clean)—no live API in tests.

7.3 Repository structure

`corpus.json` (frozen v1 roots, pins, OSV snapshot), `sbom.py` (pin parsing), `osv.py` (minimal OSV client + fixtures), `reach.py` (AST import/usage verdicts), `report.py` (one-command table), `tests/` (the runnable check), `internals/` (working notes and audit log, not published except this report).

References

- [1] OSV.dev API, POST `/v1/query`. <https://google.github.io/osv.dev/post-v1-query/>
- [2] Go team. *Tutorial: Find and fix vulnerable dependencies with govulncheck*. <https://tip.golang.org/doc/tutorial/govulncheck>
- [3] osv-scanner PR #2131: determine package reachability level of Python projects. <https://github.com/google/osv-scanner/pull/2131>
- [4] Scanner-consistency study (Grype vs Trivy vs DepScan). <https://arxiv.org/html/2503.14388v3>
- [5] pypa/pip-audit (reference OSV client). <https://github.com/pypa/pip-audit>
- [6] OpenVEX: Vulnerability Exploitability Exchange (output-format context). <https://openvex.dev>
- [7] urllib3 GHSA-2xpw-w6gg-jr37 (streaming decompression, in range for 1.26.18). <https://github.com/urllib3/urllib3/security/advisories/GHSA-2xpw-w6gg-jr37>
- [8] Werkzeug GHSA-29vq-49wr-vm6x (safe_join devices, in range for 2.0.3). <https://github.com/pallets/werkzeug/security/advisories/GHSA-29vq-49wr-vm6x>
- [9] PYSEC-2026-2132 / CVE-2026-7246 (click.edit() injection, in range for 8.0.4). <https://api.osv.dev/v1/vulns/PYSEC-2026-2132>